COMPUTER VISION FOR ROBOTIC OBJECT SORTING

Robotics Autonomy 2026

Problem Statement 2

Introduction :

I have builds a transparent OpenCV baseline that turns a tabletop image into tool proposals, shape labels, image-space centers, orientation estimates, and candidate grasp points. I have explained in this report that what the method can show and where it fails, especially under occlusion and without calibration.

End-to-end workflow



1. Image preprocessing

Grayscale. An RGB/BGR image stores three color channels. A grayscale image reduces each pixel to one intensity value, which simplifies intensity gradients and edge detection and reduces computation. Grayscale discards hue information, so the project retains the color image for segmentation and converts a blurred copy to grayscale for Canny edge display.

Scaling and resizing. Resizing can standardize image dimensions and reduce runtime, but it can also remove small details or make thin tools disappear. Use area interpolation for shrinking and linear/cubic interpolation for enlarging. Preserve the x/y scale factors if measurements must be mapped back to original pixels. Image gradients (for example, Sobel derivatives) estimate the rate of intensity change horizontally and vertically; their magnitude is large at many object boundaries. They can also respond to texture and lighting transitions.

Noise reduction. Gaussian blur uses a weighted neighborhood and reduces high-frequency noise before gradients are calculated. Median filtering replaces a pixel with a neighborhood median and is useful for isolated salt-and-pepper noise. A bilateral filter can smooth while preserving some strong edges, at higher computational cost. Excessive smoothing can erase small jaws, graduations, or cable boundaries, so the kernel should be checked visually.

2. Edge detection

Canny first smooths the image, estimates intensity gradients and their directions, thins candidate edges with non-maximum suppression, and uses low/high thresholds with hysteresis to keep weak edges connected to strong edges. Other methods include Sobel and Scharr first derivatives, Laplacian second derivatives, and Prewitt gradients. An edge map makes transitions visible and can support boundary extraction, but it contains no semantic label: shadows, printed markings, reflections, and touching tools can all create edges. The included plot shows Canny edges for inspection; candidate contours are extracted from foreground masks, which are more closed than raw edge traces in the generated examples.

3. Contours and shape detection

A contour is an ordered curve tracing a boundary in a binary image. In a typical OpenCV workflow, thresholding, foreground segmentation, or edge detection produces a binary image and `findContours` traces connected boundaries. Retrieval mode determines whether nested boundaries are retained; approximation mode controls how many points describe each contour. The project uses external contours of cleaned foreground masks, so holes are not treated as separate objects.

Contour tracing turns boundary pixels into a geometric object that can be measured. Useful descriptors include area, perimeter, bounding-box aspect ratio, convex hull, solidity, polygon approximation, centroid from image moments, and Hu moments. A circle has high circularity, a rectangular body has a compact polygon approximation, and a triangle has three dominant vertices. Real tools are not simple primitives, so shape evidence should be combined with color, dimensions, and (when available) known templates.

Template matching logic. Convert a template and the search image to a consistent representation (often grayscale or an edge map), evaluate a similarity measure such as normalized correlation at candidate locations, and retain strong local maxima after non-maximum suppression. To tolerate orientation or scale changes, search rotated and resized template variants. The project includes a normalized-correlation helper for that workflow and uses a separate rotation-searching silhouette comparison for its simple shape labels. Both methods can confuse similar tools and handle occlusion poorly; a template score is a candidate measure, not proof of identity.

4. Coordinate conversion and localization

An image point `(u, v)` is measured in pixels, usually with the origin at the top-left and x increasing rightward/y downward. A robot target is expressed in a robot-base or world frame, with physical units and a different origin/orientation. For a fixed camera viewing a flat table, four or more non-collinear image/table correspondences can estimate a planar homography `H`; homogeneous coordinates are mapped by `q = H [u, v, 1]^T`, then divided by the third coordinate. Additional points allow a least-squares estimate and residual checks. The table-to-robot rigid transform then maps the planar location into the robot base frame. A non-planar scene or moving camera requires camera intrinsics, camera-to-robot (hand-eye) calibration, and depth or 3D geometry. Calibration requires measured correspondences; pixel coordinates alone are not physical robot coordinates.

Polar coordinates can represent planar position relative to the robot as distance `r = sqrt(x^2 + y^2)` and bearing `theta = atan2(y, x)`. This is convenient for radial reach and direction reasoning, though Cartesian coordinates remain useful for arm kinematics.

TOOL-SORTING TASK DESIGN

1. **Acquire and inspect:** use a fixed overhead camera, steady illumination, a matte table, and a known image scale. Inspect several dataset images, including rotated tools and overlap, before tuning thresholds. Check annotations and class names before measuring performance.

2. **Preprocess:** retain color, optionally resize, apply a modest Gaussian/median filter, and create grayscale and HSV views. Correct uneven lighting if needed; do not discard color when it separates tools from the table.

3. **Segment candidates:** use background subtraction, HSV color proposals, or GrabCut to form foreground regions. Apply morphological opening to remove small noise and closing to bridge small gaps. For touching objects, marker-based watershed can split a merged foreground region when its seeds are good; poor seeds may over-segment or merge objects. Use Canny as a boundary cue and compare it with the foreground mask.

4. **Extract geometry:** find external contours, reject tiny regions, and calculate moments, bounding boxes, aspect ratio, area, solidity, orientation, and shape descriptors.

5. **Recognize:** compare multiple cues (contour shape, dimensions, color, and optional rotated/scaled templates). Keep an unknown/uncertain class when evidence is weak. The included demo uses HSV regions to propose candidates and then compares normalized silhouettes to three rotated reference masks. A separate GrabCut mode can form candidates on a mostly uniform background. This is a small hand-built baseline, not a trained or general tool classifier; ambiguous and occluded crops may get the wrong label.

6. **Handle touching tools:** a single binary foreground mask may merge two touching objects. Try class/color masks when colors differ; otherwise marker-controlled watershed or instance segmentation can separate candidates. Partial occlusion cannot recover hidden geometry with certainty; report visible evidence and confidence rather than inventing it.

7. **Choose a grasp candidate:** use the centroid or oriented bounding box as a first visualization. For a real gripper, choose a collision-free point on a stable, sufficiently wide section, account for the tool's orientation and gripper width, and check that the proposed pose is reachable. A centroid is not automatically a safe grasp point (for example, it may fall in the open jaw of a wrench).

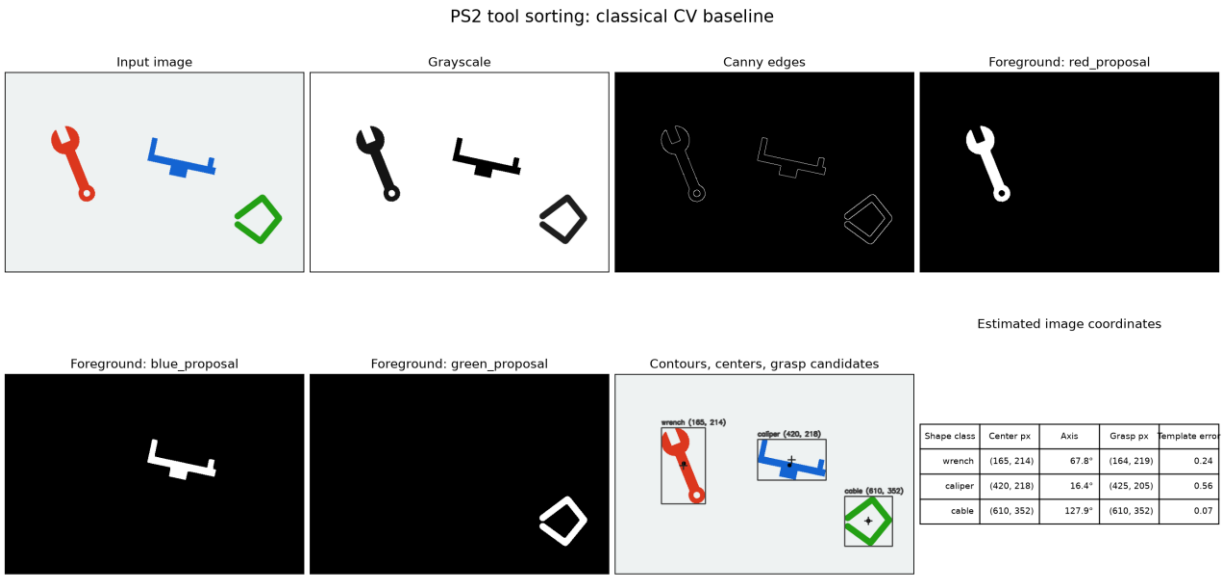
8. **Map coordinates:** report `(u, v)` and candidate angle first. With a calibration matrix, map to table coordinates and then to the robot base; without calibration, leave coordinates in pixels and do not command a robot.

9. **Evaluate and iterate:** inspect correct detections, missed objects, merged contours, false positives, and grasp-point quality on both easy and difficult images. Change one threshold or kernel at a time and save intermediate outputs.

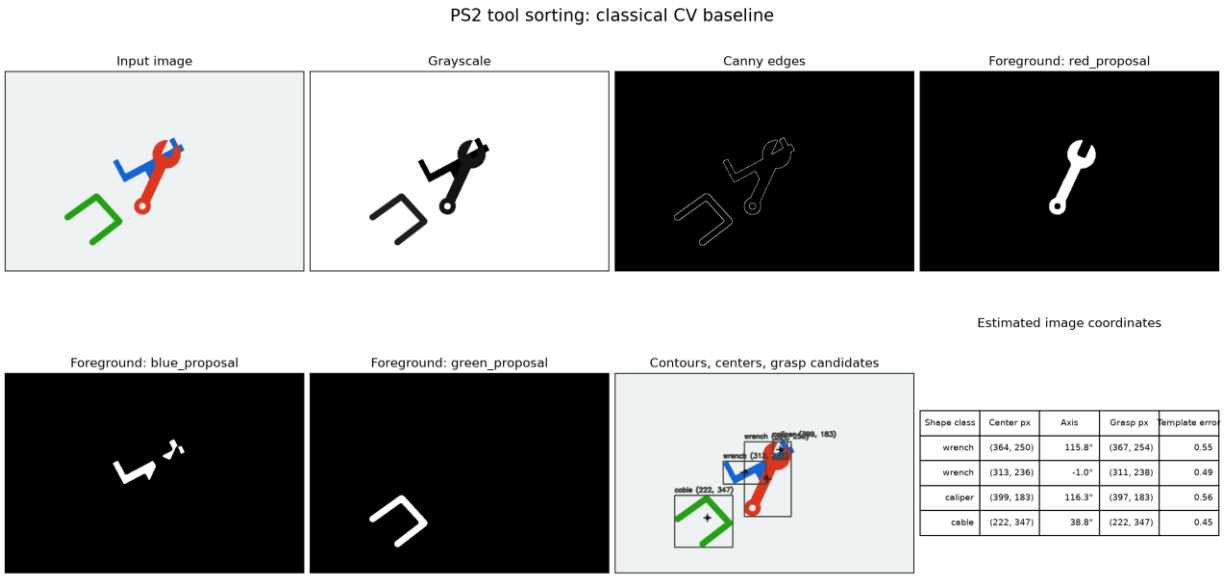
DEMONSTRATION AND OBSERVATIONS

The repository includes two **generated illustrative images**, not downloaded dataset examples. On the separated scene, the current settings return one wrench, one caliper, and one cable candidate. On the overlap scene, they return four contours: the partially covered caliper is split into two visible blue regions, so four contours do not mean four physical tools. The detector reports each retained color contour independently and cannot infer the hidden object boundary. These examples verify code paths and visualization only; they are not an accuracy evaluation on the Roboflow dataset.

The dataset linked by the assignment is published by Nguyen Phuc Thinh on Roboflow Universe under CC BY 4.0. Its page lists ten object labels, including wrench and caliper; the assignment's examples also mention cables, which need not correspond to a dataset class. The page showed no downloadable dataset version during this review. Before claiming performance on that dataset, obtain a compatible image subset, record its source/license, inspect its annotations, and test it with documented thresholds and labels.



Generated illustration: separated tools.



Generated illustration: overlapping tools.

Limitations and safety

The demo's HSV ranges are only candidate proposals for generated images; real-image GrabCut can still confuse tools with a changing or textured background. The three shape templates cover only three silhouettes and can misclassify differently designed tools. A rotated/scaled template search is slower and cannot restore hidden edges. Watershed can over-segment. The displayed template error is a geometric similarity distance, not calibrated confidence. The code estimates image-plane candidates only. Real robot use needs camera/robot calibration, depth or a verified planar setup, reachability checks, collision checking, gripper constraints, and safety supervision. No robot motion is commanded.

References

1. OpenCV, “Canny Edge Detector,” https://docs.opencv.org/4.10.0/da/d5c/tutorial_canny_detector.html

2. OpenCV, “Image Gradients,” https://docs.opencv.org/4.11.0/d5/d0f/tutorial_py_gradients.html

3. OpenCV, “Smoothing Images,” https://docs.opencv.org/4.x/dc/dd3/tutorial_gausian_median_blur_bilateral_filter.html

4. OpenCV, “Changing Colorspaces,” https://docs.opencv.org/4.x/df/d9d/tutorial_py_colorspaces.html

5. OpenCV, “Morphological Transformations,” https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html

6. OpenCV, “Finding contours in your image,” https://docs.opencv.org/4.x/df/d0d/tutorial_find_contours.html

7. OpenCV, “Image Moments,” https://docs.opencv.org/4.x/d0/d49/tutorial_moments.html

8. OpenCV, “Template Matching,” https://docs.opencv.org/4.11.0/d4/dc6/tutorial_py_template_matching.html

9. OpenCV, “Image Segmentation with Watershed Algorithm,” https://docs.opencv.org/4.6.0/d3/db4/tutorial_py_watershed.html

10. OpenCV, “Geometric Transformations of Images,” https://docs.opencv.org/4.x/da/d6e/tutorial_py_geometric_transformations.html

11. Nguyen Phuc Thinh, “mechanical tools Dataset,” Roboflow Universe, CC BY 4.0, https://universe.roboflow.com/nguyen-phuc-thinh/mechanical-tools-t59rw

12. Robotics, “Problem Statement 2: Computer Vision for Robotic Object Sorting,” supplied assignment PDF.